CONSOLIDATED TESTING PROCEDURE AND DOCUMENTATION

Backend services — RentaMóvil

RENTAMOVIL

Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez Vargas

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

1. CHANGE CONTROL

Version	Date	Description	Owner
1.0	2026-10-16	Consolidation of the procedure, inventory and test results of the six Java backend modules, including `rtm-telemetry-gps` and the standardization of tests to English.	Development team

2. OBJECTIVE

To establish a common, repeatable and verifiable procedure for designing, implementing, running and recording the tests of RentaMóvil's backend services. The document consolidates the current inventory, the results per module, the observed limitations and the recommended actions.

Existing, verified capabilities are documented. Suggested tests are presented as pending improvements, not as coverage that has already been implemented.

3. SCOPE

This document covers the six Java backend modules:

- `rtm-iam`: identity and access.
- `rtm-fleet-maintenance`: fleet and maintenance.
- `rtm-booking-reservation`: reservations and rentals.
- `rtm-payment-billing`: payments and billing.
- `rtm-api-gateway`: request entry point and routing.
- `rtm-telemetry-gps`: GPS telemetry and tracking of rentals in progress.

It includes domain and use-case unit tests, direct tests of web components/filters, two JPA repository suites with H2 (Fleet and Telemetry GPS), and a separate reference to the manual integration verification with Postman.

It does not certify all HTTP endpoints, full SQL Server compatibility, contracts between services, performance, comprehensive security, or automated E2E flows. The web and mobile applications and the database migration repositories are not part of this automated baseline.

4. TEST COMPONENTS AND ENVIRONMENT

Module	Java	Spring Boot	Production persistence
`rtm-iam`	21	4.1.1	JPA / SQL Server
`rtm-fleet-maintenance`	21	4.1.1	JPA / SQL Server
`rtm-booking-reservation`	21	4.1.1	JPA / SQL Server
`rtm-payment-billing`	21	4.1.1	JPA / SQL Server

Module	Java	Spring Boot	Production persistence
`rtm-api-gateway`	21	4.0.8	Does not manage business persistence
`rtm-telemetry-gps`	21	4.1.1	JPA / SQL Server

The main verified tool versions are Java 21 and Maven 3.9.16. The modules use JUnit Jupiter through `spring-boot-starter-test`; Mockito and AssertJ appear in the existing tests. Fleet and Telemetry GPS also add H2 and Spring Boot 4's JPA test module in `test` scope (`spring-boot-starter-data-jpa-test` in Fleet, `spring-boot-data-jpa-test` in Telemetry GPS; both resolve to the same functionality — in Spring Boot 4.x `@DataJpaTest` was moved to a separate module and is no longer included in `spring-boot-starter-test`).

Run with JDK 21, which is the version declared in the POMs. In this environment, a run with JDK 27 installed as the system default failed during initialization of the Lombok annotation processor (`ExceptionInInitializerError` on `EndPosTable`); the baseline was obtained by explicitly setting `JAVA_HOME` to JDK 21 on each run, without changing the operating system's default JDK.

5. TESTING STRATEGY

5.1 Domain and use cases

Test business rules with JUnit 5, isolate external dependencies with test doubles where appropriate, and verify results, errors and relevant interactions. Do not start Spring for unit tests of logic that can be tested directly.

The tests of all six modules follow a common language standard: method names, `@DisplayName` annotations and comments in English, regardless of whether a module's production code is in Spanish (this is the case for `rtm-iam`, `rtm-booking-reservation`, `rtm-payment-billing`, `rtm-api-gateway` and `rtm-telemetry-gps`; only `rtm-fleet-maintenance` also has its production code in English). The only deliberate exception is assertions that literally verify the text of an exception or notification message generated by the production code (for example `.hasMessageContaining("posterior")` in `rtm-booking-reservation`): that fragment is left in Spanish because it is exactly what the real code produces, and translating it would invalidate the test. The production code itself was not translated; that is a separate, pending decision.

5.2 Controllers and filters

Some tests invoke controller methods directly or test filters as isolated units. They are useful for validating delegation and local logic, but they do not go through the MVC server and, on their own, do not verify HTTP routes, JSON serialization, input validation or integrated HTTP handling.

No `@WebMvcTest`/`MockMvc` tests were found in the six modules. If HTTP contracts need to be validated, add focused MVC tests.

5.3 Persistence

Fleet and Telemetry GPS each contain a `@DataJpaTest` suite with H2. Fleet covers queries on branches, vehicles, maintenance records and models. Telemetry GPS covers the GPS device catalog and the two `@Query` location queries (`findLatest`, `findRoute`) that restrict the search to each rental's time window — the core mechanism of `Location.INV-001`/`INV-002`. H2 validates JPA and JPQL behavior, but it does not replace SQL Server nor cover all of its particularities.

IAM, Booking and Billing use JPA in production, but no `@DataJpaTest`, H2 or Testcontainers tests were found in their current suites. Their repositories, mappings and queries are not considered automatically verified by this baseline.

Fleet's JPA suite passes, although Hibernate logs a warning when trying to create `shared.audit` because the shared schema is not present in H2; auditing and that schema are outside the suite's coverage. The Telemetry GPS suite avoids that warning because its tests do not set a database schema (`hibernate.default_schema` empty in `src/test/resources/application.properties`), at the cost of not validating the real production `telemetry` schema.

5.4 System integration

Fleet's guide documents a manual verification with Docker, SQL Server, gateway and Postman: 37 of 37 requests successful on October 4, 2026. It is dated manual evidence; it is not part of `mvn test` and has not been repeated in later automated runs. No equivalent manual verification is documented for `rtm-telemetry-gps` as of this baseline.

6. PROCEDURE

6.1 Analyze the change

1. Inspect the POM, the package structure, the source classes and nearby tests.
2. Identify the existing behavior, the contracts, the dependencies and the actual exceptions.
3. Select the appropriate test level; do not invent entities, endpoints or responses.
4. Determine the limits of what the test can demonstrate, especially if it uses mocks or an embedded database.

6.2 Implement tests

1. Place tests under `src/test/java`, respecting the package of the code under test.
2. Isolate use cases from Spring and use mocks for external dependencies where appropriate.
3. Test web components directly only for their local logic; use `MockMvc` if HTTP guarantees are required.
4. For JPA repositories, use a persistence slice with minimal data and document the difference from SQL Server.
5. Cover successful paths, alternative paths and errors relevant to the change, with assertions on values and interactions.

6. Write method names, `@DisplayName` and comments in English, except for assertions that literally verify a real production-code message (see 5.1).

6.3 Run from the rentacar root

The repositories are independent Maven projects; there is no common aggregator POM. Run each module separately:

```
mvn -f rtm-iam/pom.xml test
mvn -f rtm-fleet-maintenance/pom.xml test
mvn -f rtm-booking-reservation/pom.xml test
mvn -f rtm-payment-billing/pom.xml test
mvn -f rtm-api-gateway/pom.xml test
mvn -f rtm-telemetry-gps/pom.xml test
```

If the system's default JDK is not version 21 as declared in the POMs, explicitly set `JAVA_HOME` before each command, for example:

```
JAVA_HOME=/usr/lib/jvm/java-21-openjdk mvn -f rtm-telemetry-gps/pom.xml test
```

To run a specific suite, add `-Dtest=TestName` before `test`. Maven Surefire reports are left in `target/surefire-reports` inside each module.

6.4 Evaluate and record

1. Confirm compilation and `BUILD SUCCESS`.
2. Record the number of tests, failures, errors and skipped tests per module.
3. Investigate initialization warnings even if they do not change the suite's result.
4. Record the date, Java, command, branch/commit and scope of the test.
5. Separate automated evidence from manual tests and describe their limits.

7. INVENTORY AND RESULTS PER MODULE

Module	Inventory and behavior exercised	Result verified on 2026-10-16
`rtm-iam`	Eleven test classes, focused on use cases: registration, login/logout, tokens, profile, email, password, roles and status.	39 tests; 0 failures; 0 errors; `BUILD SUCCESS`
`rtm-fleet-maintenance`	Fifteen use-case classes, six controllers invoked directly, JWT verification and a `@DataJpaTest` suite with H2 for branches, vehicles, maintenance records and models.	116 tests; 0 failures; 0 errors; 0 skipped; `BUILD SUCCESS`
`rtm-booking-reservation`	Reservation/rental/insurance domain, payment events, expiration and controllers invoked directly. Two in-memory repositories are test helpers, not suites.	53 tests; 0 failures; 0 errors; `BUILD SUCCESS`
`rtm-payment-billing`	Two test classes: `Payment` entity and `PaymentUseCaseImpl` use case.	14 tests; 0 failures; 0 errors; `BUILD SUCCESS`
`rtm-api-gateway`	Three test classes: JWT filter, correlation ID filter and in-memory rate limiter.	10 tests; 0 failures; 0 errors; `BUILD SUCCESS`
`rtm-telemetry-gps`	Three use cases (GPS catalog, location/tracking, reactions to rental events), five controllers invoked directly, RS256 JWT verification, device API key validator, in-memory tracking, and persistence with `@DataJpaTest`/H2 plus adapters tested with Mockito.	57 tests; 0 failures; 0 errors; `BUILD SUCCESS`
Total Java backend	**Independent runs per module**	**289 tests; 0 failures; 0 errors**

The total of 289 is an aggregate of six independent Maven runs, not a multi-module reactor run.

8. ACCEPTANCE CRITERIA

- The affected module compiles with Java 21 and its full suite finishes without failures or errors.
- Test names and scenarios correspond to existing classes and behavior.
- Tests are repeatable and do not depend on remote services or real credentials, except in controlled manual procedures.
- A slice test declares the database engine and the differences from production.
- Manual checks are not presented as automated.
- No coverage percentage is reported if no measurement is configured.
- New and existing tests are written in English (method names, `@DisplayName`, comments), except for assertions that literally verify a real production-code message in Spanish.

9. KNOWN GAPS AND RISKS

Priority	Gap	Affected scope
High	Missing real HTTP validation with `@WebMvcTest`/`MockMvc`; direct tests do not test transport or serialization.	All six modules; especially Fleet, Booking and Telemetry GPS controllers
High	There are no repository tests in three JPA modules; Fleet and Telemetry GPS use H2, not SQL Server, and do not test `shared.audit` or the real `telemetry` schema.	IAM, Booking, Billing and the limits of Fleet and Telemetry GPS
High	Billing has two test classes and 14 cases, the smallest inventory per service.	Billing
Medium	There are no automated tests of the Gateway's routes/upstreams in an HTTP context.	API Gateway
Medium	JaCoCo is not configured, so there is no reproducible coverage percentage.	All
Medium	There is no CI pipeline configured to run the suites when integrating changes.	All
Medium	The integrated Postman verification is manual, not repeatable through `mvn test`, and no equivalent exists for Telemetry GPS.	Entire system; Telemetry GPS in particular
Low	Production code remains in Spanish in five of the six modules, while the tests are already in English; the language inconsistency between test and tested code may confuse anyone unaware of the decision.	IAM, Booking, Billing, API Gateway, Telemetry GPS

10. RECOMMENDED IMPROVEMENT ACTIONS

1. Prioritize business-rule and payment-error tests in Billing.
2. Add MVC tests with `MockMvc` for endpoints and HTTP error handling.
3. Add persistence tests for IAM, Booking and Billing, replicating the `@DataJpaTest` + H2 pattern of Fleet and Telemetry GPS; evaluate Testcontainers with SQL Server for real compatibility.
4. Complete tests of the Gateway's routing and upstreams.
5. Configure JaCoCo with agreed targets and add `mvn test` to CI per repository.
6. Gradually automate the most critical Postman/E2E flows; document a manual integration verification for Telemetry GPS equivalent to Fleet's.
7. Decide whether the production code is translated to English to remove the language inconsistency between test and tested code, or whether Spanish is kept permanently as the production standard.

These actions are proposals; they are not part of the results executed here.

11. ROLES AND RESPONSIBILITIES

Role	Responsibility
Developer	Implement the tests for the change, run them in their module and record the results.
Technical reviewer	Confirm that the scenarios correspond to the real code and that their limits are stated.
Quality lead or manager	Approve acceptance criteria and prioritize the gaps.
Environment owner	Maintain the services and test data required for manual or integrated validations.